

Extend or Repair? Vocabulary Extension Cannot Cross a Pre-Tokenizer Boundary

Aarjan Chaudhary and Shrey Sharma and Navyata Dhakal

Independent researchers, Nepal

arjanchaudhary@gmail.com

Abstract

Many widely used open LLMs split text with a regular expression that treats a word as a run of Unicode letters. Devanagari vowel signs are combining marks, not letters, so this letters-only regex cuts every Nepali word apart before tokenization, and Nepali costs current open models that use it $2.6\text{--}5.8\times$ as many tokens as English. Vocabulary extension, the standard way to adapt a deployed model to a new language, cannot cross these cuts: for Llama-3 and Qwen3 no number of appended merges takes the Nepali premium below about $2.2\times$, and ten released extensions of Llama-3 and Qwen models for six Indic and South-east Asian scripts already sit within 4% of their floors. We give a two-substitution repair of the deployed regex that provably leaves the token ids of every input with no combining mark and no Devanagari code point unchanged, which includes all ASCII English and code. Combined with continued-BPE extension, it brings Nepali to $1.01\times$ English at 32,000 new merges and $0.95\times$ at 64,000, and gives the same pattern for seven other languages, with no change for two languages whose scripts have no combining marks. In controlled continued pretraining of Llama-3.2-1B on 2 GB of text, extension of either kind is worse on held-out Nepali than keeping the original vocabulary. Between the two extensions, the repaired one is within 0.006 bits per byte of the standard one, with 26% less training compute, slightly better English, and $1.48\times$ faster Nepali generation. We release the tokenizers, a diagnostic tool, code and pre-registration.

1 Introduction

A Nepali sentence costs a deployed open LLM several times more tokens than its English translation: $4.42\times$ under the Qwen3 tokenizer and $4.76\times$ under the c1100k tokenizer shared by Phi-4, OLMo-2 and Granite-4.1, measured on parallel FLORES-200 text (§3). Every extra token is paid for in

money, latency and context length, so the premium is a recurring cost for the tens of millions of people who speak Nepali.

A large share of this premium has a known, mechanical cause. GPT-2-style pre-tokenizers split text with a regular expression whose word class is $\backslash p\{L\}$, one or more Unicode *letters* (OpenAI, 2019). We call this the *letters-only regex*. Devanagari writes its dependent vowel signs, virama, anusvara and candrabindu as combining *marks* ($\backslash p\{M\}$), so the regex starts a new pre-token at every vowel sign, and byte-pair encoding can never merge across that boundary. Regmi et al. (2026b) formalised this as a training-free lower bound on fertility. They showed that widening the word class to $[\backslash p\{L\}\backslash p\{M\}]^+$ cuts Nepali from 4.78 to 1.58 tokens per word, and found the letters-only class in 63% of the 1,345 Hugging Face text-generation repositories they classified. OpenAI’s o200k tokenizer and Meta’s Llama-4 already ship a mark-aware class.

That result concerns tokenizers trained from scratch. Practitioners rarely train from scratch: the standard way to adapt a deployed model to a new language is *vocabulary extension*, which adds target-language tokens, initialises their embeddings and continues pretraining (Tejaswi et al., 2024; Yamaguchi et al., 2026; Purason et al., 2026; Smith et al., 2026). This paper asks what happens when that recipe meets the pre-tokenizer.

Vocabulary extension cannot cross a pre-tokenizer boundary. New merges, like old ones, act only within pre-tokens. A model whose regex shatters Nepali into P pre-tokens can never encode it in fewer than P tokens, however many merges are appended. Released extensions already sit on this bound: ten published extensions of Llama-3 and Qwen models for six scripts written with combining marks use only 1.3–3.6% more tokens than their pre-token count (Figure 3). The resulting

lower bound on the Nepali/English premium, the *pre-token floor* (defined in §3), is $2.19\times$ for Llama-3 and $2.17\times$ for Qwen3 (§4). Llama-3 already sits at $2.58\times$, so the standard recipe can remove at most 15% of its current premium. The floor is also above the premium of o200k ($1.61\times$), so no vocabulary extension can bring a letters-only model to parity with GPT-4o’s tokenizer on Nepali. The three published extension studies of these model families for Indic and other abugida languages that we inspected kept the letters-only regex (Yamaguchi et al., 2026; Purason et al., 2026; Smith et al., 2026); any expansion of LFM2 ($5.77\times$ on Nepali) keeps a floor of $2.15\times$.

An English-lossless repair. The fix is to change the regex of the deployed model. Changing a pre-trained model’s pre-tokenizer usually changes how it tokenizes everything, including English (Dagan et al., 2024). We use a minimal edit, the *repaired regex*, that does not. For any input with no combining mark and no code point in U+0900–U+097F, which includes all ASCII text, the repaired tokenizer produces exactly the original token ids (Proposition 1). We then learn new merges by continued BPE (Purason et al., 2026) restricted to Devanagari outputs. The repaired tokenizers give identical token ids on all 1,012 FLORES devtest sentences in English, Russian and Amharic, and before training the resized model’s logits over the original vocabulary match the base model’s up to float error.

Contributions.

1. An audit of 33 released tokenizers (24 distinct), with each premium decomposed into a regex factor and a vocabulary factor, and the pre-token floor of every letters-only tokenizer (§3).
2. Evidence that the floor binds in practice: released extensions and our own continued BPE under the letters-only regex stall at it (§4, §7.1).
3. A minimal, provably English-lossless repair of a deployed model’s pre-tokenizer (§5).
4. A controlled comparison of R0 (no extension), R1 (extend) and R2 (repair + extend) on Llama-3.2-1B and Qwen3-0.6B-Base under identical data, merge budget and optimization, scored per byte (§7).

5. Released tokenizers, code and pre-registration.¹

2 Related work

Tokenization premiums. Tokenizers trained mostly on English encode other languages in more tokens, so those languages pay more for the same content (Petrov et al., 2023; Ahia et al., 2023; Lundin et al., 2026; Ovcharov, 2026; Somide, 2026; Tamang and Bora, 2024). For Nepali, Kadariya (2026) report $4.79\times$ for c1100k. Srivastava (2026) and Mandarapu and Kunkunuru (2026) measure large Indic premiums on FLORES-200, and the former reports that o200k cuts the mean Indic premium from $8.0\times$ to $2.1\times$, without attributing the change to the regex. We report sentence-level token ratios on parallel FLORES-200 text (Goyal et al., 2022; NLLB Team et al., 2022), following Petrov et al. (2023), with paired bootstrap intervals. Fewer tokens do not by themselves make a better model (Schmidt et al., 2024), which is why we test the repair with continued pretraining.

Pre-tokenization of combining marks. A 2024 tiktoken issue reported that GPT-2-style regexes split Indic words at vowel signs and proposed the word class $[\backslash p\{L\}\backslash p\{M\}]^+$ (ajaykg, 2024). ZeTT (Minixhofer et al., 2024) adjusted its GPT-2-based regex to avoid over-segmenting Hindi and Tamil. Velayuthan and Sarveswaran (2025) showed that the GPT-2, GPT-4 and Llama-3 pre-tokenizers break Tamil, Sinhala and Hindi text, and Rana et al. (2025) gained 38–40% in fertility by swapping the GPT-2 regex for Llama-4’s. Regmi et al. (2026b) identified the cause (marks fall outside $\backslash p\{L\}$), stated the pre-token lower bound (their pre-tokenization ceiling) across 26 languages, and showed with 268M-parameter models trained from scratch that the wider class lowers Nepali bits per byte by 4.4%. They found the letters-only class in 63% of 1,345 Hugging Face text-generation repositories, carrying 72.5% of 30-day downloads, and note that o200k already ships the repair. Land (2026) lists the families still affected and reports that Qwen3.5 fixed it. The edit itself is known (tiktoken #292, ZeTT, o200k, Qwen3.5); what is not studied is repairing a deployed model.

¹Pre-registration commits are timestamped before any continued-pretraining result was observed; see Appendix A.

Beyond one character class. Grapheme- and akshara-aware pre-tokenizers enforce syllable boundaries (Kapila and Bagavathy, 2026; Darshana, 2026; Krishnan K and Santhiappan, 2026; Shrestha and Pradhan, 2026), MorphTok (Brahma et al., 2025) constrains BPE merges, and SuperBPE and BoundlessBPE (Liu et al., 2025; Schmidt et al., 2025) lift pre-token boundaries in a second training stage. All of this work trains new tokenizers; we study a model that already exists.

Adapting the vocabulary of a pretrained model.

Vocabulary extension adds tokens, initialises their embeddings from existing ones (Gee et al., 2022; Minixhofer et al., 2022; Dobler and de Melo, 2023; Liu et al., 2024; Mundra et al., 2024) and continues pretraining (Tejaswi et al., 2024; Kim et al., 2024; Csaki et al., 2024); Duwal et al. (2024) continue pretraining Llama-3-8B on Nepali without new tokens, like our R0. On byte-level models, continued BPE appends new merges to the existing list (Purason et al., 2026; Smith et al., 2026). Yamaguchi et al. (2026) extend Llama-3 and Qwen models for several abugidas. These studies cover our model families but keep the original pre-tokenizer, which caps them (§4). ZeTT applies its mark-aware regex to target tokenizers for SentencePiece models, which have no regex to repair. Dagan et al. (2024) change the pre-tokenizer of Code Llama during fine-tuning and find that recovery takes more than 50B tokens; their change alters English and code tokenization, which ours does not.

The closest prior system is TituLLM (Nahin et al., 2025), which extends Llama-3.2 for Bangla with a \w-based regex that keeps vowel signs attached. The paper does not identify the regex as a cause and has no regex-only or vocabulary-only arm, and its regex also changes digit, identifier and whitespace pre-tokens in English. On Nepali its premium (2.60×) is slightly above Llama-3’s (2.58×): a mark-aware regex without target-language merges does not lower the premium by itself. We give a controlled attribution, and a repair that leaves every input without marks or Devanagari code points unchanged.

3 Production audit

We measured 33 released tokenizers on parallel FLORES-200 devtest text. Nine have identical token counts to another tokenizer on all 36 FLORES languages we measured (for example, Phi-

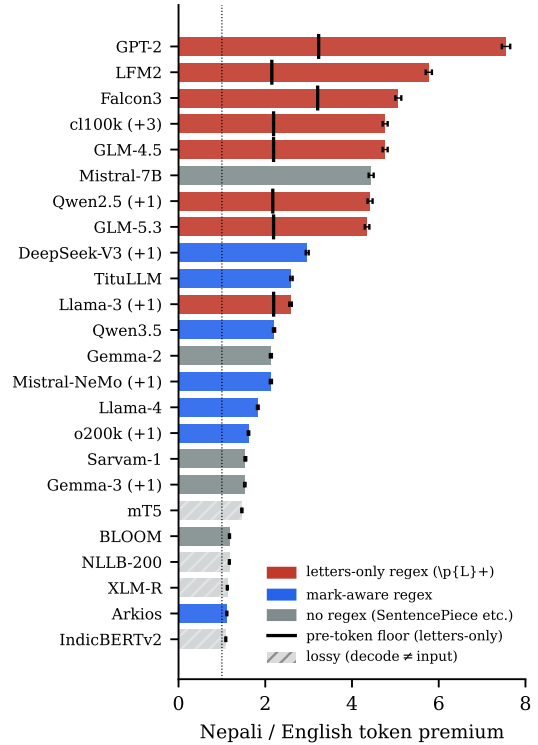


Figure 1: Nepali/English token premium of 24 distinct tokenizers (33 released tokenizers; “+n” counts aliases) on FLORES-200 devtest, with 95% paired-bootstrap intervals. Black ticks mark the pre-token floor of each letters-only tokenizer: no vocabulary extension can move its bar to the left of the tick.

4, OLMo-2 and Granite-4.1 all match cl100k; Qwen3 matches Qwen2.5; SmolLM3 matches Llama-3). We treat them as aliases, leaving 24 distinct tokenizers (Figure 1; full table in Appendix F). We classify each tokenizer by its own pre-tokenizer configuration, not by family. Four SentencePiece and WordPiece tokenizers (XML-R, mT5, NLLB-200, IndicBERTv2) do not reproduce their input on decoding, so their low premiums are not comparable compression results; Figure 1 marks them, and we exclude them from the analysis below.

The pre-token floor. Let P be a tokenizer’s pre-tokenizer and T the tokenizer. Summed over the 1,012 devtest sentences, we call $|P(\text{ne})|/|T(\text{en})|$ the *pre-token floor* of T . The Nepali/English premium cannot go below it under any vocabulary extension that keeps the pre-tokenizer and leaves English tokenization unchanged (§4). It is a lower bound, not a value that extension is guaranteed to reach.

Every letters-only tokenizer has a floor of at least $2.15\times$. Eight distinct tokenizers, covering 13 released names including GLM-5.3 and LFM2, use the letters-only class. Their premiums range from $2.58\times$ (Llama-3) to $7.55\times$ (GPT-2), and their pre-token floors from $2.15\times$ to $3.23\times$. Excluding GPT-2, a 2019 model that we include as a historical reference, the premiums of current letters-only models range from $2.58\times$ to $5.77\times$ (LFM2). The seven lossless tokenizers with a mark-aware regex range from $1.12\times$ (Arkios; Regmi et al., 2026a) to $2.97\times$ (DeepSeek-V3).

Decomposition. We write each premium as a product of three factors (Appendix G): a base term, which lies between 0.77 and 0.80 for every tokenizer; a *regex factor*, the ratio of Nepali pre-tokens under the tokenizer’s own regex to those under the repaired regex; and a *vocabulary factor*, tokens per pre-token, which is the only factor vocabulary extension can lower, and only down to 1. The regex factor is 2.77–2.79 for the Llama-3/Qwen/c1100k pattern, 4.06 for the GPT-2 and Falcon3 patterns, and 0.98–1.02 for mark-aware tokenizers. For Llama-3 the vocabulary factor is 1.18, the lowest in the audit, so almost all of its remaining premium is the regex factor. Among mark-aware tokenizers it ranges from 1.43 (Arkios) to 3.79 (DeepSeek-V3): fixing the regex is necessary but not sufficient.

Within-family changes. From Qwen3 to Qwen3.5 the Nepali premium halves ($4.42\times$ to $2.20\times$), with a word branch close to our repaired class (Land, 2026); from Llama-3 to Llama-4, which adopts o200k’s pattern, it falls from $2.58\times$ to $1.83\times$. The vocabulary also grew in both cases, so these corroborate, but do not replace, the controlled comparison of §7.1.

Broken syllables. Many Nepali tokens begin with a combining mark, which no well-formed Devanagari syllable does: 55% of Llama-3’s, and still 26% of o200k’s, because BPE also splits within words (Appendix F). §7.1 shows how the rate moves under each retrofit.

4 The pre-token floor binds vocabulary extension

Pre-tokens bound tokens. A byte-level BPE tokenizer first splits text s with a regex into pre-tokens $P(s) = (p_1, \dots, p_m)$, then encodes each

letters-only $\backslash p\{L\}+$	न	े	प	ाल	को	सर	कार
repaired $[\backslash p\{L\}\backslash p\{M\}]^+$	नेपालको		सरकार				

Figure 2: Pre-tokens of *nepālko sarkār* (“government of Nepal”) under the letters-only regex of Llama-3 and Qwen and under the repaired regex. Dotted circles mark pre-tokens that begin with a dependent vowel sign.

p_i independently by applying merges in rank order. Since every non-empty pre-token yields at least one token, $|T(s)| \geq |P(s)|$, a bound Regmi et al. (2026b) call the pre-tokenization ceiling. Vocabulary extension appends merges but leaves P unchanged, so the same bound holds for every extended tokenizer. Strings registered as HuggingFace *added tokens* bypass the bound (Appendix I).

Observation 1 (Extension floor; a direct corollary of the bound of Regmi et al., 2026b). *Let T_K be any tokenizer obtained from T by appending K merges, for any K and any set of merges regardless of training data, without changing its normalizer or pre-tokenizer and without added tokens. Then $|T_K(s)| \geq |P(s)|$ for every string s .*

To turn this into a floor on the premium, we divide by the English token count of the *base* tokenizer. This step needs the extension to leave English unchanged. It does for R1: every new merge outputs a token containing a code point in U+0900–U+097F, so no new merge can apply to English text, by the argument of Proposition 1 with the regex left unchanged. The premium of R1 is therefore at least the pre-token floor of §3, for every K .

The bound is not loose for the regexes deployed today. In the Llama-3 and Qwen families, the word branch is $[\wedge r \backslash n \backslash p\{L\} \backslash p\{N\}]^? \backslash p\{L\}^+$: a run of letters, optionally preceded by one non-letter. A Devanagari vowel sign is not a letter, so it ends the current run and is absorbed as the optional prefix of the next one. In Figure 2, the phrase *nepālko sarkār* (“government of Nepal”) becomes six pre-tokens, four of which begin with a vowel sign that belongs to the *previous* consonant. Under the repaired regex each word is one pre-token.

The floor, measured. Table 1 gives the pre-token floor of Llama-3 and Qwen3 under their own letters-only regexes and under the repaired regex of each model. For both families the letters-only

	Llama-3	Qwen3	o200k
Current premium	2.58	4.42	1.61
Floor, letters-only regex	2.19	2.17	–
Floor, repaired regex	0.79	0.80	0.80

Table 1: Nepali/English token premium on FLORES-200 devtest, and the pre-token floor under each regex (pre-tokens of Nepali / tokens of English): a lower bound on the premium of any vocabulary extension under that regex. Each model’s floors use its own pattern (Qwen splits digits singly, Llama-3 in groups of up to three). Llama-3 and Qwen3 ship the letters-only regex; for o200k, which ships a mark-aware regex, the floor is under its own pattern.

floor is about $2.2\times$, nearly three times the repaired floor ($2.77\times$ for Llama-3, $2.73\times$ for Qwen3), and above the premium of o200k ($1.61\times$).

Released extensions sit on the floor. Figure 3 and Table 6 (Appendix I) measure 12 released vocabulary extensions of letters-only models (15 extension–language pairs): eleven by Yamaguchi et al. (2026) on Llama-3, Llama-3.1, Qwen2.5 and Qwen3, ten of them for six scripts written with combining marks and one for Amharic, and the 61k-token in-place expansion of LFM2 (Smith et al., 2026), on four of its languages. Every Yamaguchi extension for a script with combining marks uses $1.013\times$ to $1.036\times$ as many tokens as its floor, closing 98.2–99.5% of the gap between the base tokenizer and the floor. LFM2.5 sits at $1.07\times$ its floor for Hindi and $1.16\times$ for Nepali. Under the repaired regex the same text has 2.2 to 5.9 times fewer pre-tokens, so these extensions would have had that much more room. Yamaguchi’s Amharic extension, for a script without combining marks, stays at $1.57\times$ its floor, and the repair does not move that floor. Two releases fall below the letters-only floor: TituLLM (Nahin et al., 2025) changed the regex (§2), and SinLlama (Aravinda et al., 2025) registered 11,080 strings as added tokens.

5 Repairing a deployed tokenizer

The repair has three parts: a one-class edit to the regex, new merges learned on top of the existing merge list, and an embedding initialisation that leaves the model’s computation on old tokens unchanged.

Regex edit. We admit $\backslash p\{M\}$ wherever the regex has a letter word class, and exclude it from the op-

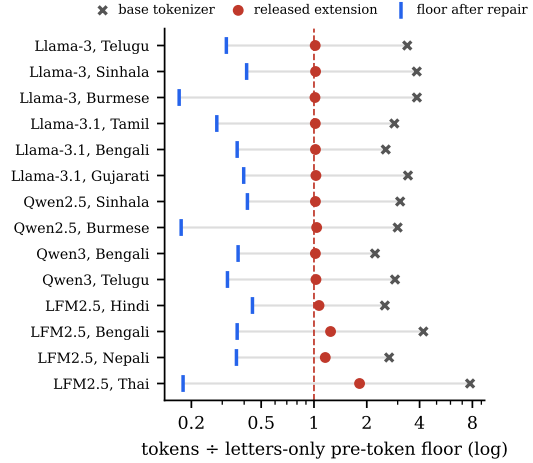


Figure 3: Released vocabulary extensions of letters-only models, on FLORES-200 devtest of each target language, relative to the letters-only pre-token floor. Ten extensions by Yamaguchi et al. (2026) (top) and LFM2.5 (Smith et al., 2026) (bottom) move from the base tokenizer ($\times$) to the floor ($\bullet$) and stop there. The repaired regex would lower the floor to the blue tick. Full numbers, including the Amharic control, in Table 6.

tional leading class and the punctuation class, so that a mark can only be matched by the word class and stays in one pre-token with the letters around it. For the Llama-3/Qwen pattern this is two substitutions (Appendix B). Digits, contractions, whitespace and punctuation are handled as before. We call the result the *repaired regex*. Qwen3.5 independently shipped almost the same word branch (§3; Land, 2026).

Continued BPE, restricted to Devanagari.

Following Purason et al. (2026), we learn K new merges on top of the existing ones. We pre-tokenize Nepali text with the arm’s regex, segment every pre-token with the *original* merges, as the deployed model sees it, and run standard BPE over these segmentations, treating each base token as an indivisible symbol. We append the new merges after the existing ones, so the original merges act first at inference. We discard any merge whose output contains no code point in U+0900–U+097F, together with merges that depend on it. Two edge cases and implementation details are in Appendix B. R1 and R2 use this identical procedure on identical text with identical K ; they differ only in the regex.

Proposition 1 (English invariance). *Let s be a string, after the tokenizer’s normalizer, containing*

no code point of category \p{M} and no code point in U+0900–U+097F. Then the repaired tokenizer R2 and the original tokenizer produce identical token ids for s.

The proof is in Appendix C. The proposition covers ASCII English and code, and every other script whose normalised text has no combining marks; it does not cover Devanagari or text with marks (Appendix B).

Embedding initialisation. Each new token’s input and output rows are set to the mean of the rows of its constituent base tokens (Gee et al., 2022), obtained by applying the original merges to its byte-level string. Original rows are untouched (Qwen3’s first new tokens reuse its unused padding rows). On any input whose ids are all original, the resized model therefore gives the same logits for every original token as the base model; we check this on an English probe sentence (maximum absolute difference $< 10^{-4}$). The softmax denominator gains the new tokens’ logits, so continued pretraining is still needed.

Arms. R0 (no extension) keeps the original tokenizer. R1 (extend) appends K merges learned under the original letters-only regex: the standard recipe (Purason et al., 2026; Yamaguchi et al., 2026). R2 (repair + extend) switches to the repaired regex and appends K merges learned under it. All arms then continue pretraining on the same bytes (§6).

6 Experimental setup

This section follows the second pre-registration amendment (Appendix A), which reduced the registered design to fit our budget before any continued-pretraining result was observed. Full details are in Appendix D.

Models. Llama-3.2-1B (base) and Qwen3-0.6B-Base, two widely used open models whose tokenizers ship the letters-only regex (§3). Both tie input and output embeddings. Qwen3-0.6B-Base shares the tokenizer of Qwen3-1.7B-Base, on which we built the Qwen tokenizers of §7.1. The base models are evaluated as a reference.

Data and tokenizers. Nepali text is FineWeb-2 np_i_Deva; English is FineWeb-Edu (sample-10BT). We normalise to NFC, deduplicate, and remove every document that shares a whitespace 10-gram with FLORES-200 dev or

devtest. A 30 MB held-out slice of each language is never trained on. R1 and R2 learn merges on the same 1 GB of Nepali text. We sweep $K \in \{1, 2, 4, 8, 16, 32, 64\} \times 10^3$ for compression (§7.1) and use the pre-registered $K=32,000$ for training.

Continued pretraining. Each arm is trained for one epoch on the *same documents*: 1 GB of Nepali and 1 GB of English, interleaved in one fixed order. Because R2 encodes the same bytes in fewer tokens, it takes fewer steps and less compute. For an equal-compute view, R0 and R1 also save a checkpoint at the step count where R2 ends, before their cosine schedules have annealed. Optimisation is identical across arms, with a fixed, untuned peak learning rate of 10^{-4} and one seed (seed 0) per arm.

Evaluation. Every metric is normalised by UTF-8 bytes, never by tokens. The primary metric is *byte-matched* bits per byte (BPB) on held-out native Nepali: each document is split at whitespace into pieces of about 2,000 bytes, scored independently, so every arm conditions on the same bytes. Secondary metrics are held-out English BPB, sliding-window and FLORES-200 BPB, zero-shot Belebele (Bandarkar et al., 2024), 5-shot FLORES translation (chrF++) and Nepali generation speed in bytes per second.

Analysis. The primary comparison is R2 against R1 and against R0 on byte-matched held-out Nepali BPB, at equal data (final checkpoints). We use a paired bootstrap over held-out pieces ($B=10,000$) of the per-piece BPB difference, and report the mean difference with its 95% interval. We call R2 better if the interval of (R2 – other) lies entirely below 0, non-inferior if it lies entirely below +0.01 bits per byte, and otherwise report no detectable difference. The same test on English pieces is the English control (R2 against R0). With one seed, we cannot separate arm effects from data-order variance.

7 Results

This section reports the controlled experiments: compression of the extended tokenizers and continued pretraining.

7.1 Compression of repaired tokenizers

Figure 4 sweeps the number of new merges K for both models. Under the letters-only regex (R1),

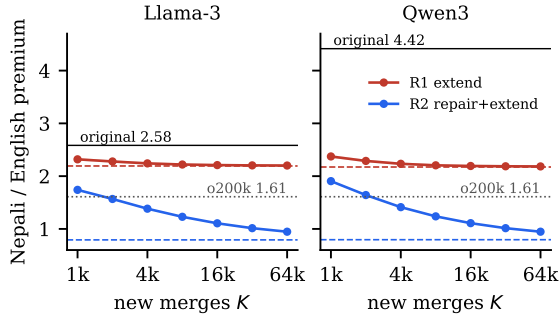


Figure 4: Nepali/English premium on FLORES-200 devtest after adding K merges by continued BPE, under the letters-only regex (R1) and the repaired regex (R2). Dashed lines: pre-token floors of each regex, under each model’s own pattern. Qwen3 tokenizers are built on Qwen3-1.7B-Base, whose tokenizer Qwen3-0.6B-Base shares. Shading: 95% paired-bootstrap intervals.

extension saturates almost at once: for Llama-3 the first 1,000 merges take the premium from $2.58\times$ to $2.32\times$, and the next 63,000 only to $2.20\times$, within 0.01 of the pre-token floor ($2.19\times$). For Qwen3, R1 cuts the premium from $4.42\times$ to $2.19\times$ by $K=16k$ and then stops at $2.18\times$ against a floor of $2.17\times$. That R1 comes this close to its floor, rather than merely staying above it, is consistent with the small number of distinct Nepali pre-tokens the letters-only regex produces: the 1 GB merge-training text has about 124k distinct pre-tokens under it, against 1.54M under the repaired regex. Under the repaired regex (R2), the same merge budget keeps paying: $1.11\times$ at $K=16k$, $1.01\times$ at 32k and $0.95\times$ at 64k for both models, below parity with English and below o200k. At the pre-registered $K=32,000$, R2 encodes Nepali in 46% of R1’s tokens for both models.

All 28 extended tokenizers are lossless on FLORES Nepali and English and give token ids identical to the original tokenizer on all 1,012 English devtest sentences, as Proposition 1 (for R2) and its argument in §4 (for R1) require. FLORES dev, which we used to check the choice of K , gives the same picture within 0.02.

Regex alone, and added tokens. Two further arms separate the parts of R2. Repairing the regex without adding merges (R2, $K=0$) changes almost nothing: $2.61\times$ for Llama-3 (from $2.58\times$) and $4.42\times$ for Qwen3. Under the repaired regex, the old merges still tokenize Nepali words in old pieces; the regex only removes the ceiling that

new merges would otherwise hit. Registering R1’s 32,000 new strings as added tokens instead of merges gives $2.35\times$ for Llama-3 and $2.19\times$ for Qwen3, no better than R1. The gain of R2 therefore needs both the regex repair and new merges.

Broken syllables. Extension under the letters-only regex makes syllable breaking worse. The share of Devanagari tokens that begin with a combining mark rises from 57% (Llama-3) and 33% (Qwen3) to 67% for both under R1, because new merges can only build on mark-initial fragments. Under R2 it falls to 15% and 12%.

Other scripts. We repeated the sweep for eight more languages, using the FineWeb-2 test split of each (25–361 MB) to learn merges, with the same Devanagari-style output filter adapted to each script (Appendix H). Table 5 (Appendix H) gives the result at $K=32,000$; for example, R2 takes Tamil from $7.65\times$ to $1.02\times$ and Burmese from $11.62\times$ to $1.21\times$ on Llama-3, where R1 stops at $2.67\times$ and $3.02\times$. For every language written with combining marks, R1 stops at its floor and R2 reaches 0.88 – $1.24\times$. For the two scripts without combining marks, Amharic and Korean, R1 and R2 are identical at every K , as the mechanism predicts. All 126 tokenizers in this sweep are lossless and give English token ids identical to their base tokenizer on all 1,012 English devtest sentences.

7.2 Continued pretraining

Table 2 reports every run. We state each comparison with the pre-registered paired bootstrap over held-out Nepali pieces (difference in bits per byte, 95% interval).

Extension of either kind costs quality at this budget. For Llama-3.2-1B, continued pretraining without new tokens (R0) lowers held-out Nepali BPB from 0.766 to 0.474. Both extended arms end higher: R1 by $+0.060$ [$+0.059$, $+0.061$] and R2 by $+0.066$ [$+0.065$, $+0.067$] bits per byte. The cost does not come from the repair: it appears equally when 32,000 tokens are added under the original regex. The 32,000 new rows start from averages of existing rows; with 1 GB of Nepali they do not catch up with the original vocabulary, which the base model already knows. R0 is also best at English-to-Nepali translation (chrF++ 19.8 against 10.4 and 12.3).

Model	Data	Arm	Tokens	ne BPB↓	en BPB↓	FLORES ne↓	Belebele ne	chrF ne→en	chrF en→ne	Gen. B/s
Llama-3.2-1B	–	base	0	0.766	0.788	0.863	0.277	32.4	10.8	5327
	1 GB	R0	417M	0.474	0.799	0.611	0.287	32.3	19.8	5147
	1 GB	R1	386M	0.534	0.803	0.648	0.268	27.2	10.4	6354
	1 GB	R2	286M	0.539	0.799	0.671	0.266	27.8	12.3	9398
Qwen3-0.6B	–	base	0	0.852	0.873	0.945	0.290	26.0	6.8	2186

Table 2: Continued pretraining on identical bytes per arm (one seed). BPB: bits per byte on held-out native text split into $\sim 2,000$ -byte pieces (primary metric) and on FLORES-200 devtest. Belebele: zero-shot accuracy (900 items, chance 0.25), scored on answer text. chrF++: 5-shot FLORES-200 devtest translation. Gen. B/s: UTF-8 bytes of Nepali generated per second (batch 64, greedy). Tokens: training tokens seen. Bold: best Nepali BPB within each block.

Among extensions, the repair matches standard extension with less compute. R2 and R1 differ by $+0.006$ $[+0.005, +0.007]$ bits per byte on Nepali, inside the pre-registered non-inferiority margin of 0.01, although the interval excludes zero. R2 reaches this with 286M training tokens against R1’s 386M (26% fewer), and is slightly better on English (-0.003 $[-0.004, -0.003]$). At equal compute, when R1 is stopped after the same number of steps as R2, R2 is better on Nepali (0.539 against 0.562), though R1’s learning rate has not yet annealed at that point. R2 generates Nepali at 9,398 bytes per second against 6,354 for R1 and 5,147 for R0, $1.48\times$ and $1.83\times$ faster, because each token carries 2.5 times more text than under R0.

English is unaffected. English BPB after continued pretraining is 0.799 for both R0 and R2 ($+0.0003$ $[-0.0001, +0.0007]$), as the invariance result predicts for the tokenizer and as the equal English data in every arm predicts for the model.

8 Discussion

Practical recommendation. Anyone adapting a Llama-3, Qwen2.5/3, GLM, Phi-4, OLMo-2, Granite or LFM2 model to Nepali, or to another language written with combining marks, should check the pre-tokenizer before extending the vocabulary. The check takes one line: count pre-tokens on a target-language sample under the model’s regex and compare with the token count. If they are close, as for Llama-3 on Nepali or the Telugu-extended Llama-3 of §4, extension has little room left. The repair in §5 is a two-substitution edit that leaves the token ids of all ASCII English and code unchanged (Proposition 1). Deployments whose tokenizer has no normalizer, such as Llama-3, should apply NFC before tokenizing to

keep accented Latin text inside the guarantee.

What the repair does not fix. A wider word class removes the splits the regex forces; it does not make BPE respect syllables, and 26% of o200k’s Nepali tokens still begin with a vowel sign (§3). Grapheme- and akshara-aware tokenizers (Kapila and Bagavathy, 2026; Darshana, 2026; Velayuthan and Sarveswaran, 2025) go further, but they change more than marks and would not preserve English tokenization in a deployed model. Combining the two in a retrofit is future work.

Limitations

- **Scale and budget.** We adapt 1B- and 0.6B-parameter models with 2 GB of text per arm (1 GB per language). Larger models, or much longer continued pretraining, may close part of the gap between arms; Dagan et al. (2024) found that a model recovers from a pre-tokenizer change after more than 50B tokens of training, far more than we use.
- **One seed.** Each arm is trained once. We cannot separate arm effects from data-order variance, and we say so wherever we make a claim.
- **Learning rate.** A single fixed learning rate (10^{-4}) is used for every arm and is not tuned. The new embedding rows of R1 and R2 might prefer a different rate or a separate warm-up stage.
- **Equal compute.** The equal-compute checkpoints of R0 and R1 are taken mid-schedule, before their learning rate has annealed, which disadvantages them.
- **Downstream tasks.** Bits per byte is our primary metric. Belebele and few-shot translation are weak signals at this model size, and

we report them as secondary.

- **Embedding initialisation.** We use mean initialisation for all arms. Stronger initialisers (Dobler and de Melo, 2023; Minixhofer et al., 2024) could change the absolute numbers, though they would apply to R1 and R2 alike.
- **Languages.** Continued pretraining is done for Nepali only. For eight other languages we show the compression effect and the floors of released extensions, but not the effect on model quality.
- **Closed tokenizers.** We do not measure Claude or Gemini API tokenizers, whose regexes are not public.

Ethics and use of AI assistance

All text and evaluation data are public (FineWeb-2, FineWeb-Edu, FLORES-200, Belebele) and used under their licences; we do not collect data from people. Released Llama-derived weights follow the Llama 3.2 Community License (Meta Platforms, Inc., 2024); Qwen3-derived weights are Apache-2.0.

This work was carried out with substantial assistance from Claude (Anthropic), an AI system, which under the authors’ direction implemented and ran the experiments, searched and verified the literature, and drafted the text. The numbers in the paper can be reproduced from public tokenizers and data with our released code. The authors take responsibility for the content.

References

- Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David Mortensen, Noah Smith, and Yulia Tsvetkov. 2023. [Do all languages cost the same? tokenization in the era of commercial language models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9904–9923, Singapore. Association for Computational Linguistics.
- ajaykg. 2024. [Combining marks and indic vowel marks within words are being split breaking all indic languages and most languages except english and CJKs \(tiktoken issue #292\)](#). GitHub issue, openai/tiktoken.
- H. W. K. Aravinda, Rashad Sirajudeen, Samith Karunathilake, Nisansa de Silva, Rishemjit Kaur, and Surangika Ranathunga. 2025. [SinLlama – a large language model for Sinhala](#). In 2025 *Moratuwa Engineering Research Conference (MER-Con)*, pages 617–622. IEEE.
- Lucas Bandarkar, Davis Liang, Benjamin Muller, Mikel Artetxe, Satya Narayan Shukla, Donald Husa, Naman Goyal, Abhinandan Krishnan, Luke Zettlemoyer, and Madian Khabza. 2024. [The belebele benchmark: a parallel reading comprehension dataset in 122 language variants](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 749–775, Bangkok, Thailand. Association for Computational Linguistics.
- Maharaj Brahma, N J Karthika, Atul Singh, Devaraj Adiga, Smruti Bhate, Ganesh Ramakrishnan, Rohit Saluja, and Maunendra Sankar Desarkar. 2025. [MorphTok: Morphologically grounded tokenization for Indian languages](#). *Preprint*, arXiv:2504.10335. Tokenization Workshop (TokShop) at ICML 2025.
- Zoltan Csaki, Bo Li, Jonathan Li, Qiantong Xu, Pian Pawakapan, Leon Zhang, Yun Du, Hengyu Zhao, Changran Hu, and Urmish Thakker. 2024. [SambaLingo: Teaching large language models new languages](#). *Preprint*, arXiv:2404.05829.
- Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. 2024. [Getting the most out of your tokenizer for pre-training and domain adaptation](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 9784–9805.
- Kusal Darshana. 2026. [Separate before you compress: The WHO tokenization architecture](#). *Preprint*, arXiv:2603.25309.
- Konstantin Dobler and Gerard de Melo. 2023. [FOCUS: Effective embedding initialization for monolingual specialization of multilingual models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13440–13454, Singapore. Association for Computational Linguistics.
- Sharad Duwal, Suraj Prasai, and Suresh Manandhar. 2024. [Domain-adaptive continual learning for low-resource tasks: Evaluation on Nepali](#). *Preprint*, arXiv:2412.13860.
- Leonidas Gee, Andrea Zugarini, Leonardo Rigutini, and Paolo Torroni. 2022. [Fast vocabulary transfer for language model compression](#). In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 409–416, Abu Dhabi, UAE. Association for Computational Linguistics.
- Naman Goyal, Cynthia Gao, Vishrav Chaudhary, Peng-Jen Chen, Guillaume Wenzek, Da Ju, Sanjana Krishnan, Marc’Aurelio Ranzato, Francisco Guzmán, and Angela Fan. 2022. [The Flores-101 evaluation benchmark for low-resource and multilingual machine translation](#). *Transactions of the Association for Computational Linguistics*, 10:522–538.

- Ramesh Kadariya. 2026. How well does AI understand Nepali? a multi-task benchmark of large language models and embedding models on a low-resource language. Preprint (Zenodo/ResearchGate). Code: <https://github.com/RameshKadariya/nepali-llm-benchmark>. UN-VERIFIED: exact date and DOI.
- Sai Hemanth Kapila and Rakshika Bagavathy. 2026. [Type-driven tokenization for Brahmic scripts](#). Preprint, arXiv:2609.22125.
- Seungduk Kim, Seungtaek Choi, and Myeongho Jeong. 2024. [Efficient and effective vocabulary expansion towards multilingual large language models](#). Preprint, arXiv:2402.14714.
- Hari Krishnan K and Sudarsun Santhiappan. 2026. [A grapheme-aware indic tokenizer for Tamil: Large-scale training and intrinsic evaluation](#). Preprint, arXiv:2609.06690.
- Sander Land. 2026. [Small pre-tokenization bugs with a big multilingual price](#). Substack blog post, 1 September 2026.
- Alisa Liu, Jonathan Hayase, Valentin Hofmann, Seungwon Oh, Noah A. Smith, and Yejin Choi. 2025. [SuperBPE: Space travel for language models](#). In *Conference on Language Modeling (COLM)*.
- Yihong Liu, Peiqin Lin, Mingyang Wang, and Hinrich Schuetze. 2024. [OFA: A framework of initializing unseen subword embeddings for efficient large-scale multilingual continued pretraining](#). In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 1067–1097, Mexico City, Mexico. Association for Computational Linguistics.
- Jessica M. Lundin, Ada Zhang, Nihal Karim, Hamza Louzan, Guohao Wei, David Ifeoluwa Adelani, and Cody Carroll. 2026. [The token tax: Systematic bias in multilingual tokenization](#). In *Proceedings of the 7th Workshop on African Natural Language Processing (AfricanNLP 2026)*, pages 103–112, Rabat, Morocco. Association for Computational Linguistics.
- Madhulatha Mandarapu and Sandeep Kunkunuru. 2026. [Removable and irreducible: A token-cost ledger for the multilingual tokenization tax](#). Preprint, arXiv:2609.00378.
- Meta Platforms, Inc. 2024. Llama 3.2 Community License Agreement. https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/LICENSE. Version release date 25 Sep 2024; accessed 2026-10-04.
- Benjamin Minixhofer, Fabian Paischer, and Navid Reksabsaz. 2022. [WECHSEL: Effective initialization of subword embeddings for cross-lingual transfer of monolingual language models](#). In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3992–4006, Seattle, United States. Association for Computational Linguistics.
- Benjamin Minixhofer, Edoardo Maria Ponti, and Ivan Vulić. 2024. [Zero-shot tokenizer transfer](#). In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 46791–46818. Appendix I: target pre-tokenizer is the GPT-2 regex adjusted for the Unicode Mark category (code: SPLIT_REGEX in <https://github.com/bminixhofer/zett/blob/main/zett/utis.py>).
- Nandini Mundra, Aditya Nanda Kishore Khandavally, Raj Dabre, Ratish Puduppully, Anoop Kunchukuttan, and Mitesh M Khapra. 2024. [An empirical comparison of vocabulary expansion and initialization approaches for language models](#). In *Proceedings of the 28th Conference on Computational Natural Language Learning*, pages 84–104, Miami, FL, USA. Association for Computational Linguistics.
- Shahriar Kabir Nahin, Rabindra Nath Nandi, Sagor Sarker, Quazi Sarwar Muhtaseem, Md Kowsher, Apu Chandraw Shill, Md Ibrahim, Mehadi Hasan Menon, Tareq Al Muntasir, and Firoj Alam. 2025. [TituLLMs: A family of Bangla LLMs with comprehensive benchmarking](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24922–24940, Vienna, Austria. Association for Computational Linguistics.
- NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, et al. 2022. [No language left behind: Scaling human-centered machine translation](#). arXiv preprint arXiv:2207.04672.
- OpenAI. 2019. [GPT-2 source code: src/encoder.py](#). GitHub repository openai/gpt-2.
- Volodymyr Ovcharov. 2026. [The tokenizer tax across 25 European languages: Domain invariance, cross-lingual few-shot effects, and the Ukrainian penalty](#). Preprint, arXiv:2605.24718.
- Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. 2023. [Language model tokenizers introduce unfairness between languages](#). In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.
- Taïdo Purason, Pavel Chizhov, Ivan P. Yamshchikov, and Mark Fishel. 2026. [Teaching old tokenizers new words: Efficient tokenizer adaptation for pretrained models](#). In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 6492–6516, Rabat, Morocco. Association for Computational Linguistics.
- Souvik Rana, Arul Menezes, Ashish Kulkarni, Chandra Khatri, and Shubham Agarwal. 2025. [MUTANT: A recipe for multilingual tokenizer design](#). Preprint, arXiv:2511.03237. V1 titled “IndicSuperTokenizer: An Optimized Tokenizer for Indic Multilingual LLMs”.
- Sajal Regmi, Siddhartha Pudasaini, and Chetan Phakami Pun. 2026a. [Arkios: An open bilingual english-nepali language model trained](#)

from scratch, with a devanagari-aware tokenizer. *Preprint*, arXiv:2608.30092.

Sajal Regmi, Siddhartha Pudasaini, and Chetan Phakami Pun. 2026b. [Vowel signs are not letters: A pre-tokenization ceiling on multilingual tokenizer fertility](#). *Preprint*, arXiv:2608.26449.

Craig W. Schmidt, Varshini Reddy, Chris Tanner, and Yuval Pinter. 2025. [Boundless byte pair encoding: Breaking the pre-tokenization barrier](#). In *Conference on Language Modeling (COLM)*.

Craig W Schmidt, Varshini Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. 2024. [Tokenization is more than compression](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 678–702, Miami, Florida, USA. Association for Computational Linguistics.

Kalash Shrestha and Nikhil Pradhan. 2026. [Preserving morphemes: Morphology-guided pre-tokenization for Nepali](#). *Preprint*, arXiv:2609.33395.

Jimmy T. H. Smith, Tarek Dakhraan, Alberto Cabrera, Simon S. Lee, Paul Pak, Aditya Tadimeti, Tim Seyde, Maxime Labonne, Alexander Amini, and Mathias Lechner. 2026. [In-place tokenizer expansion for pre-trained LLMs](#). *Preprint*, arXiv:2607.15232.

Olaoye Anthony Somide. 2026. [The African language tax: Quantifying the cost, latency, and context penalty of tokenizing African languages in frontier LLMs](#). *Preprint*, arXiv:2606.24460.

Priyansh Srivastava. 2026. [The tokenizer tax: Quantifying and explaining the cross-lingual cost of subword tokenization for Indian languages](#). *Preprint*, arXiv:2607.24276.

S. Tamang and D. J. Bora. 2024. [Evaluating tokenizer performance of large language models across official Indian languages](#). *Preprint*, arXiv:2411.12240.

Atula Tejaswi, Nilesh Gupta, and Eunsol Choi. 2024. [Exploring design choices for building language-specific LLMs](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10485–10500, Miami, Florida, USA. Association for Computational Linguistics.

Menan Velayuthan and Kengatharaiyer Sarveswaran. 2025. [Egalitarian language representation in language models: It all begins with tokenizers](#). In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 5987–5996, Abu Dhabi, UAE. Association for Computational Linguistics.

Atsuki Yamaguchi, Aline Villavicencio, and Nikolaos Aletras. 2026. [How can we effectively expand the vocabulary of LLMs with 0.01GB of target language text?](#) *Computational Linguistics*, 52(1):295–330.

A Pre-registration history

We pre-registered in three steps, in the public repository’s git history, and report all three.

1. Commit bf5ebe4 (2026-10-04 17:59 NPT) registered hypotheses about the mechanism, a regex-only tokenizer ablation, a cross-script dose–response and from-scratch training. Our literature check then found that [Regmi et al. \(2026b\)](#) had published the mechanism, the ablation, the cross-script analysis and a from-scratch downstream comparison five weeks earlier. We withdrew those hypotheses as contributions and do not report them as new.
2. Commit 3c03367 (2026-10-04 18:16 NPT), before any model was trained or evaluated, registered the retrofit study reported here: arms R0, R1, R2 and the untrained base model as a reference; $K=32,000$ unless R2 saturated earlier on FLORES dev; equal-bytes continued pretraining; learning rate selected once per model on R0; two seeds; primary metric held-out native Nepali BPB with a non-inferiority margin of 0.01 bits per byte; English BPB as a control; Belebele, translation and decoding speed as secondary. Its compression prediction (P1), including the letters-only floors, had already been measured when it was written, so we report P1 as a measurement, not a confirmed prediction.
3. Commit 5ff74e8 (2026-10-04 19:50 NPT), amendment 2, was written after an internal review and before any continued-pretraining run had produced an evaluation. Earlier launches of continued pretraining were stopped by us before completion because of pipeline bugs, and none produced an evaluated checkpoint. The only evaluation run before the amendment, of the untouched base Llama-3.2-1B with a since-fixed document-prefix bug, is discarded. The amendment fixed the primary analysis as stated in §6.

Deviations from the second registration (amendment 2).

- Qwen3-0.6B-Base replaces Qwen3-1.7B-Base (budget and time; same tokenizer).
- 1 GB of Nepali and 1 GB of English per arm replace 3 GB and 3 GB, with identical documents in every arm.

- A fixed learning rate of 10^{-4} for every run replaces selection on R0 from $\{3 \times 10^{-5}, 10^{-4}, 3 \times 10^{-4}\}$ by a 200-step pilot, which was never completed.
- One seed (seed 0) per arm replaces two.
- New: R0 and R1 are also checkpointed at R2's final step count (equal-compute view).
- New: byte-matched BPB on pieces of about 2,000 bytes is the primary metric; the registered sliding-window BPB becomes secondary.
- New: a paired bootstrap over held-out pieces, with explicit claim rules, is the primary test.
- The translation cap is 512 new tokens instead of 256, to avoid truncating Nepali output of R0.

Pipeline bugs fixed before any reported run.

(1) Documents had been stored one per line with their internal newlines, so “documents” were paragraphs; they are now stored as one JSON-encoded document per line. (2) New-token embeddings had been initialised by decoding tokens to text, which turns partial UTF-8 tokens into U+FFFD; constituents now come from applying the original merges to the byte-level string. (3) The initialisation exactness check had covered Qwen's padding rows; it now covers only the original vocabulary. (4) Evaluation had prefixed documents with end-of-text, while base Llama expects its beginning-of-sequence token; we now use the latter where it exists, identically for every arm of a model.

Between the first two commits, the retrofit tokenizer code was tested on a 30 MB sample of the FineWeb-2 Nepali *test* split only.

B Repair details

Regex edit. For the Llama-3/Qwen pattern the word branch changes as follows:

$$\begin{array}{c} [\text{^}\backslash\text{r}\backslash\text{n}\backslash\text{p}\{L\}\backslash\text{p}\{N\}]?\backslash\text{p}\{L\}+ \\ \downarrow \\ [\text{^}\backslash\text{r}\backslash\text{n}\backslash\text{p}\{L\}\backslash\text{p}\{M\}\backslash\text{p}\{N\}]?\backslash\text{p}\{L\}\backslash\text{p}\{M\}+ \end{array}$$

and the same change is made in the punctuation branch $[\text{^}\backslash\text{s}\backslash\text{p}\{L\}\backslash\text{p}\{N\}]+$. The optional leading class and the punctuation class must exclude $\backslash\text{p}\{M\}$, so that a mark can only be matched by the word class and stays in one pre-token with the letters around it. Nothing else in the pattern changes.

Continued BPE. At inference, BPE applies the lowest-ranked applicable merge first, so the original merges act before the new ones. The extended tokenizer reproduces the training procedure except in two cases: a new merge whose output string is already in the original vocabulary can let an original merge fire again afterwards, and under HuggingFace's `ignore_merges` option (set for Llama-3) a pre-token that is itself a vocabulary entry is emitted directly. Neither case affects Proposition 1. We run the merge learning inside the HuggingFace Rust trainer by mapping each base token to a private-use code point. Learning $K=32,000$ merges on 1 GB of Nepali text takes 1.7 to 3.4 minutes per tokenizer.

Scope of Proposition 1. The proposition covers ASCII English and code, and every other script whose normalised text has no combining marks, including most Cyrillic, Han, Hangul and Ethiopic text. It does not cover Devanagari, even without marks: mark-free Devanagari words and Devanagari digits contain code points in U+0900–U+097F and change by design, since new merges apply to them. It also does not cover text with marks, such as Hindi or Latin written in decomposed form (NFD, e.g. *café* with a combining acute). The Llama-3 tokenizer has no normalizer, so NFD input to a deployed Llama-3 R2 model falls outside the guarantee; Qwen3's tokenizer applies NFC itself, which composes most, but not all, combining marks. We normalise all our text to NFC. Empirically, R1 and R2 at $K=32,000$ give token ids identical to the original tokenizer on all 1,012 FLORES devtest sentences in English, Russian and Amharic, for both models. One Russian sentence contains a combining acute (U+0301) and so lies outside the proposition; its ids are nonetheless identical.

Embedding initialisation. A new token's constituents come from applying the original merges to its byte-level string. We do not decode it to text, which would turn tokens ending in a partial UTF-8 sequence into U+FFFD. On any input whose ids are all original, the resized model produces the same hidden states, and the same logits for every original token, as the base model.

C Proof of Proposition 1

Proof. Pre-tokens. The repaired regex differs from the original only in the three classes edited

above, and each edited class differs from its original only on $\backslash p\{M\}$ characters. A regex match on s consults only characters of s , and the order of the alternatives is unchanged. Since s has no $\backslash p\{M\}$ character, both regexes produce the same pre-tokens. *Merges and vocabulary.* R2 differs from the original tokenizer only by appended merges and by the vocabulary entries they create, each of which contains a complete code point in U+0900–U+097F. A merge (a, b) applies only where a and b are adjacent symbols of a pre-token, so its output is a contiguous substring of that pre-token’s bytes. A vocabulary entry is emitted directly under `ignore_merges` only if it equals the whole pre-token. Neither can happen for a pre-token of s , because UTF-8 is self-synchronising and s contains no such code point. Each pre-token of s is therefore encoded with original merges and original vocabulary entries only, exactly as by the original tokenizer. *Ids.* New ids are appended above the original ids, so all ids coincide. $\square$

D Training and evaluation protocol

Data. We normalise to NFC, drop documents under 200 characters, remove exact duplicates, and drop every document that shares a whitespace 10-gram with FLORES-200 dev or devtest in Nepali or English. This also covers the Belebele passages, which are FLORES passages (Bandarkar et al., 2024), but not its questions or answers. Documents are stored one JSON-encoded document per line, so internal newlines are kept within a document. Before training, a 30 MB held-out slice of each language is carved from the cleaned stream and never trained on. The 1 GB of Nepali text used to learn merges is taken from the start of the training stream. Statistics are in Appendix E.

Continued pretraining. Documents are interleaved by document and separated by the model’s end-of-text token. We report each arm’s training FLOPs. Optimisation is identical across arms: sequence length 2,048, global batch 256 sequences (0.52M tokens), AdamW ($\beta = (0.9, 0.95)$), weight decay 0.1, gradient clipping at 1.0, 1% linear warm-up, then cosine decay to 10% of peak, bf16 autocast with fp32 master weights, on $4 \times H200$. The peak learning rate is fixed at 10^{-4} for every run and is not tuned. Initialisation is deterministic and there is no dropout, so a second seed would change only the order of training windows and bf16 non-determinism.

Evaluation. Every scored sequence is prefixed by the model’s beginning-of-sequence token where it has one (Llama) and by its end-of-text token otherwise (Qwen), identically for every arm of a model.

- **Bits per byte (BPB)** on the held-out native Nepali and English slices (4 MB each), and on FLORES-200 devtest. The primary metric is byte-matched BPB: each held-out document is split at whitespace into pieces of about 2,000 bytes, and every piece is scored independently, so every arm conditions on the same bytes of context. Token windows would give R2 more bytes of context per scored token. As a secondary metric, whole documents are scored with a sliding window of 2,048 tokens and stride 1,024.
- **Belebele** (Bandarkar et al., 2024) `npi_Deva` and `eng_Latn`, zero-shot, scoring each of the four answer strings by its log-likelihood per byte given passage and question. Letter-choice prompting puts a 1B model at chance in both languages (Regmi et al., 2026a).
- **Translation:** FLORES-200 devtest Nepali $\leftrightarrow$ English, 5-shot from FLORES dev, greedy decoding capped at 512 new tokens, `chrF++`.
- **Generation speed:** UTF-8 bytes generated per second in greedy continuation of FLORES dev Nepali sentences.

The equal-compute checkpoints are scored on BPB only.

E Data

[data statistics pending]

F Full audit

Table 3 lists every distinct tokenizer. Aliases are tokenizers whose token counts are identical to another tokenizer’s on all 36 FLORES languages we measured; we did not compare token ids. Kimi-K3 (requires executing remote code) and Mistral-Large-3 (degenerate HuggingFace load) are not included.

Broken syllables. Many Nepali tokens begin with a combining mark, which no well-formed Devanagari syllable does: 55% of Llama-3’s Nepali tokens, 32% of Qwen3’s and 30% of `cl100k`’s.

Tokenizer	Aliases	Regex	$ V $	ne/en	95% CI	Floor	Mark-init.	Lossless
IndicBERTv2	–	none	250,000	1.09	[1.08, 1.10]	–	6%	no
Arkios	–	mark-aware	65,536	1.12	[1.10, 1.13]	0.78	15%	yes
XLM-R	–	none	250,002	1.12	[1.11, 1.14]	–	7%	no
NLLB-200	–	none	256,204	1.17	[1.16, 1.18]	–	6%	no
BLOOM	–	other	250,680	1.18	[1.16, 1.19]	–	17%	yes
mT5	–	none	250,100	1.46	[1.45, 1.47]	–	11%	no
Gemma-3	Gemma-4	none	262,145	1.53	[1.51, 1.54]	–	9%	yes
Sarvam-1	–	none	68,096	1.54	[1.52, 1.56]	–	14%	yes
o200k	gpt-oss	mark-aware	200,019	1.61	[1.60, 1.63]	0.80	26%	yes
Llama-4	–	mark-aware	201,135	1.83	[1.81, 1.85]	0.80	30%	yes
Mistral-NeMo	Sarvam-M	mark-aware	131,072	2.13	[2.11, 2.15]	0.79	33%	yes
Gemma-2	–	none	256,000	2.13	[2.11, 2.15]	–	19%	yes
Qwen3.5	–	mark-aware	248,077	2.20	[2.18, 2.23]	0.80	19%	yes
Llama-3	SmolLM3	letters-only	128,256	2.58	[2.55, 2.61]	2.19	55%	yes
TituLLM	–	mark-aware	170,497	2.60	[2.57, 2.63]	0.78	55%	yes
DeepSeek-V3	DeepSeek-V4	mark-aware	128,815	2.97	[2.93, 3.00]	0.78	38%	yes
GLM-5.3	–	letters-only	154,856	4.35	[4.29, 4.40]	2.19	33%	yes
Qwen2.5	Qwen3	letters-only	151,665	4.42	[4.36, 4.47]	2.17	32%	yes
Mistral-7B	–	none	32,768	4.44	[4.38, 4.50]	–	29%	yes
GLM-4.5	–	letters-only	151,365	4.76	[4.71, 4.82]	2.19	30%	yes
cl100k	Phi-4, OLMo-2, Granite-4.1	letters-only	100,277	4.76	[4.71, 4.82]	2.19	30%	yes
Falcon3	–	letters-only	131,072	5.07	[5.00, 5.14]	3.21	28%	yes
LFM2	–	letters-only	64,400	5.77	[5.70, 5.84]	2.15	24%	yes
GPT-2	–	letters-only	50,257	7.55	[7.46, 7.65]	3.23	19%	yes

Table 3: All 24 distinct tokenizers (33 released tokenizers) on FLORES-200 devtest. Aliases: tokenizers with identical token counts on all 36 FLORES languages measured. Regex: letters-only, mark-aware, other, or none (no regex pre-tokenizer). ne/en: Nepali/English token premium with 95% paired-bootstrap interval. Floor: pre-token floor under the tokenizer’s own regex, a lower bound on the premium of any vocabulary extension (letters-only and mark-aware tokenizers only). Mark-init.: share of Nepali tokens whose first character is a combining mark. Lossless: decoding reproduces the input on 200 Nepali sentences. Arkios is from [Regmi et al. \(2026a\)](#). Hugging Face repositories are listed in the released code.

The rate does not track the regex class. Mark-aware tokenizers show similar rates (26% for o200k, 55% for TituLLM, which keeps Llama-3’s merges), because BPE also splits within words.

G Premium decomposition

We write the premium as

$$\frac{|T(\text{ne})|}{|T(\text{en})|} = \underbrace{\frac{|P_{\text{rep}}(\text{ne})|}{|T(\text{en})|}}_{\text{base}} \cdot \underbrace{\frac{|P(\text{ne})|}{|P_{\text{rep}}(\text{ne})|}}_{\text{regex factor}} \cdot \underbrace{\frac{|T(\text{ne})|}{|P(\text{ne})|}}_{\text{vocabulary factor}},$$

where P is the tokenizer’s own pre-tokenizer and P_{rep} is the repaired Llama-3 pattern (§5), applied to every tokenizer. The numerator of the base term is therefore the same for all tokenizers; only the English token count varies. The base lies between 0.77 and 0.80 for every tokenizer, because English token counts barely differ. The regex factor is 0.98–1.02 for mark-aware tokenizers, 2.77–2.79 for the Llama-3/Qwen/cl100k pattern, and 4.06 for the GPT-2 and Falcon3 patterns, which put every run of marks in a pre-token of its own. The vocabulary factor, tokens per pre-token, is what vocabulary extension can lower, and only down to 1.

Table 4 shows where each tokenizer’s premium comes from. For Llama-3 the vocabulary factor is 1.18, the lowest in the audit: its 128k vocabulary already nearly covers Nepali pre-tokens, and almost all of its remaining premium is the regex factor. For Qwen and cl100k both factors matter. Among mark-aware tokenizers, the vocabulary factor varies from 1.43 (Arkios, trained for Nepali) to 3.79 (DeepSeek-V3): fixing the regex is necessary but does not by itself make a tokenizer good at Nepali. Qwen3.5’s word branch is our repaired class with only the leading-character class left unchanged; [Land \(2026\)](#) reported its change for Hindi.

H Cross-script sweep

For each language we downloaded the FineWeb-2 *test* split, normalised it to NFC, and removed documents that share a whitespace 10-gram with that language’s FLORES-200 dev or devtest (for Thai, which has no spaces, we also checked 50-character overlaps and found none). The resulting merge-learning text ranges from 25 MB (Amharic) to 361 MB (Bengali), less than the 1 GB used for

Tokenizer	Premium	Base	Regex	Vocab.
GPT-2	7.55	0.80	4.06	2.34
LFM2	5.77	0.78	2.77	2.68
c1100k (+3)	4.76	0.79	2.77	2.17
Qwen2.5/3	4.42	0.78	2.79	2.03
GLM-5.3	4.35	0.79	2.77	1.98
Llama-3	2.58	0.79	2.77	1.18
DeepSeek-V3/V4	2.97	0.80	0.98	3.79
Qwen3.5	2.20	0.78	1.02	2.76
Llama-4	1.83	0.80	1.00	2.30
o200k (+1)	1.61	0.80	1.00	2.01
Arkios	1.12	0.79	0.99	1.43

Table 4: Premium = base $\times$ regex factor $\times$ vocabulary factor, on FLORES-200 devtest, for 11 of the 15 letters-only and mark-aware tokenizers in the audit (top: letters-only; bottom: mark-aware). Arkios is from Regmi et al. (2026a). Vocabulary extension can only lower the last column, and not below 1.

Lang.	Marks	Llama-3			Qwen3		
		R0	R1	R2	R0	R1	R2
Hindi	31%	2.52	2.38	1.17	4.42	2.36	1.16
Bengali	32%	5.84	2.29	1.01	5.03	2.26	1.00
Tamil	35%	7.65	2.67	1.02	6.11	2.64	1.02
Telugu	36%	8.29	2.45	1.02	6.99	2.43	1.01
Sinhala	28%	8.63	2.30	1.20	6.88	2.33	1.24
Burmese	50%	11.62	3.02	1.21	8.93	2.99	1.11
Thai [†]	21%	2.19	1.49	0.95	2.55	1.48	0.88
Amharic	0%	7.59	0.98	0.98	4.05	0.98	0.98
Korean	0%	1.49	0.98	0.98	1.63	0.98	0.98

Table 5: Language/English token premium on FLORES-200 devtest, $K=32,000$. Marks: share of NFC code points that are combining marks. Amharic and Korean have none (control). [†]Thai is written without spaces, so whole phrases are single pre-tokens and R1 is not yet at its floor.

Nepali; Sinhala and Amharic R2 may be limited by it. $K=4k$ and $16k$ are prefixes of one $32k$ run.

The Nepali filter keeps a new merge only if its output contains a complete Devanagari character. For scripts that the base vocabularies cover poorly, many characters are themselves split into byte pieces, and this filter would drop the first byte-level merge of a character and every merge built on it (for Sinhala R2, only 13,530 of 32,000 merges survived). For the cross-script sweep we therefore also keep merges whose output is a byte piece of a target-script character. Such merges can only fire on target-script bytes, so the English guarantee is unaffected; we verified English token ids on all 1,012 sentences for every tokenizer. Hindi uses the same script as Nepali, and its new merges also change Nepali tokenization; we did not measure

that interaction.

I Released extensions

Added tokens. The bound does not apply to strings registered as HuggingFace *added tokens*, which are matched before the regex runs. SinLlama (Aravinda et al., 2025) extends Llama-3 for Sinhala this way, and thereby bypasses the bound for those exact strings. Added tokens are matched greedily and do not compose with merges, so they behave differently from a merge-based extension.

Base	Lang.	Tokens/floor	Gap closed	Room
Llama-3-8B	te	1.016	99.3%	3.16 $\times$
Llama-3-8B	si	1.021	99.3%	2.42 $\times$
Llama-3-8B	my	1.013	99.5%	5.87 $\times$
Llama-3.1-8B	ta	1.018	99.1%	3.58 $\times$
Llama-3.1-8B	bn	1.020	98.7%	2.74 $\times$
Llama-3.1-8B	gu	1.026	98.9%	2.52 $\times$
Qwen2.5-7B	si	1.019	99.1%	2.40 $\times$
Qwen2.5-7B	my	1.036	98.2%	5.72 $\times$
Qwen3-14B	bn	1.019	98.5%	2.71 $\times$
Qwen3-14B	te	1.027	98.6%	3.11 $\times$
LFM2 (2.5)	hi	1.068	95.5%	2.24 $\times$
LFM2 (2.5)	ne	1.160	90.5%	2.77 $\times$
LFM2 (2.5)	bn	1.243	92.4%	2.74 $\times$
LFM2 (2.5)	th	1.819	87.9%	5.57 $\times$
Llama-3.1-8B	am	1.573	93.8%	1.00 $\times$

Table 6: Released vocabulary extensions that keep the letters-only regex, on FLORES-200 devtest of the target language. Tokens/floor: extended-tokenizer tokens divided by pre-tokens under its own regex (at least 1 by construction). Gap closed: share of the base-to-floor gap that the extension removed. Room: floor under the letters-only regex divided by floor under the repaired regex. Top block: Yamaguchi et al. (2026); middle: LFM2.5 (Smith et al., 2026); bottom: Amharic, which has no combining marks (control).